



Batch Processing II

(v1.00)

Week 5: October 8, 2026

Jimmy Lin
David R. Cheriton School of Computer Science
University of Waterloo

These slides are available at <https://lintool.github.io/cs431-2026f/>

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International
See <https://creativecommons.org/licenses/by-nc-sa/4.0/> for details



Key Questions

In what ways does Spark improve over MapReduce?

How is distributed group by implemented efficiently at scale?

How do commutative and associative operations contribute to efficient distributed execution?

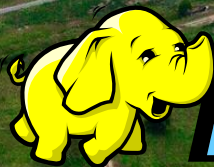
How does partitioning contribute to efficient distributed execution?

How do these concepts come together in efficient joins at scale?

The datacenter *is* the computer!

What's the instruction set?

map / reduce



hadoop





The datacenter *is* the computer!

What's the instruction set?

map / filter / flatMap ...
groupByKey / reduceByKey / join / cogroup ...

That's it.

MapReduce

```
results1 = records.map(...)  
                .reduce(...)  
  
results2 = results1.map(...)  
                .reduce(...)
```

Spark

```
results = rdd.foo(...)  
                .bar(...)  
                .baz(...)
```

Technical Deep Dives

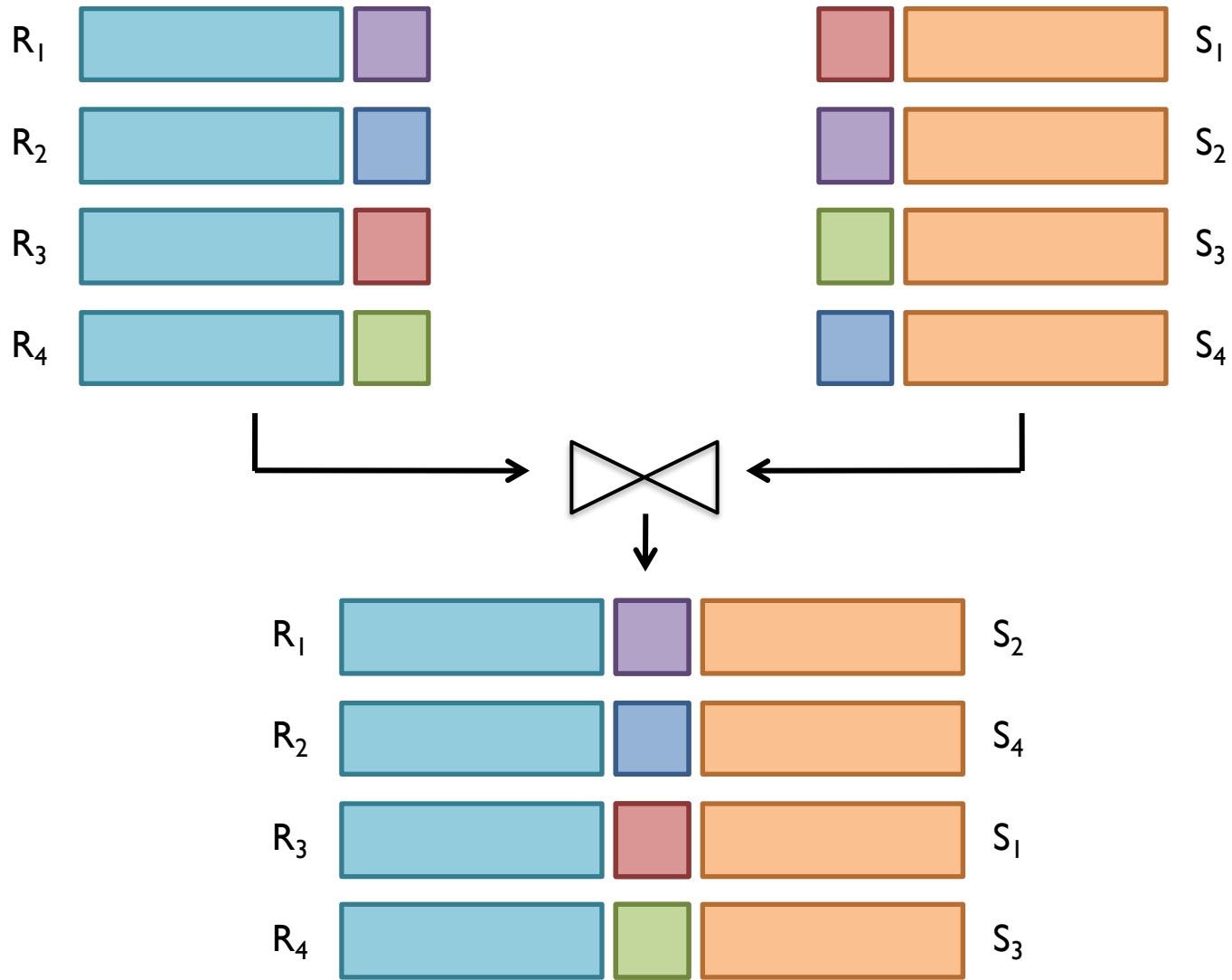
Implementation of distributed group by

... implications for algorithm design

The importance of partitioning

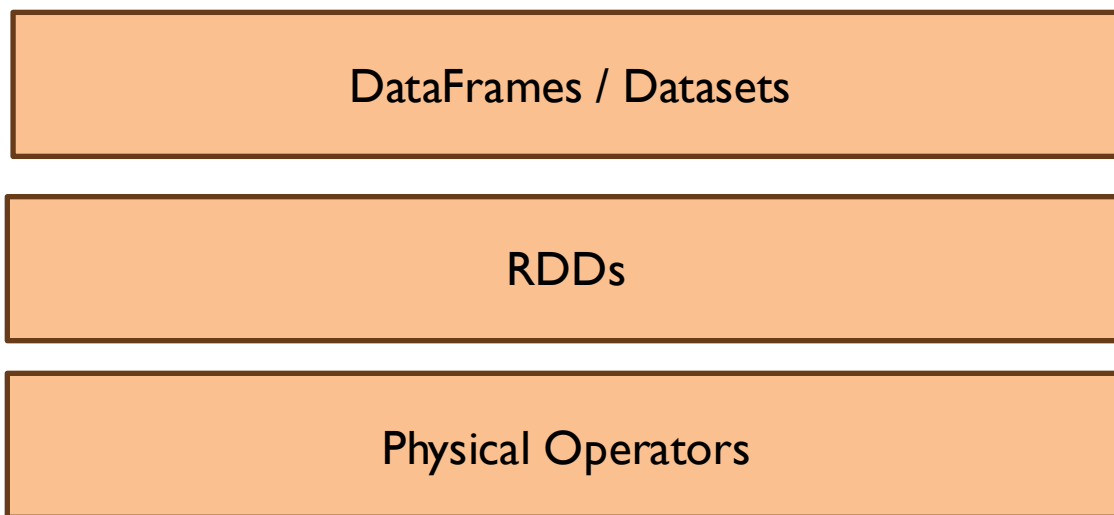
Case study in relational joins

Relational Joins



(More precisely, an inner join)

Spark Stack



What level are we operating in?

Join Algorithms in MapReduce / Spark

Reduce-side join

aka repartition join

aka shuffle join

Map-side join

aka sort-merge join

Hash join

aka broadcast join

aka replicated join

Reduce-side Join

aka repartition join, shuffle join

Intuition: group by join key

Map over both datasets

Emit tuple as value with join key as the intermediate key

Execution framework brings together tuples sharing the same key

Perform join in reducer

Reduce-side Join

aka repartition join, shuffle join

Intuition: group by join key

~~Map over both datasets~~ Union two RDDs

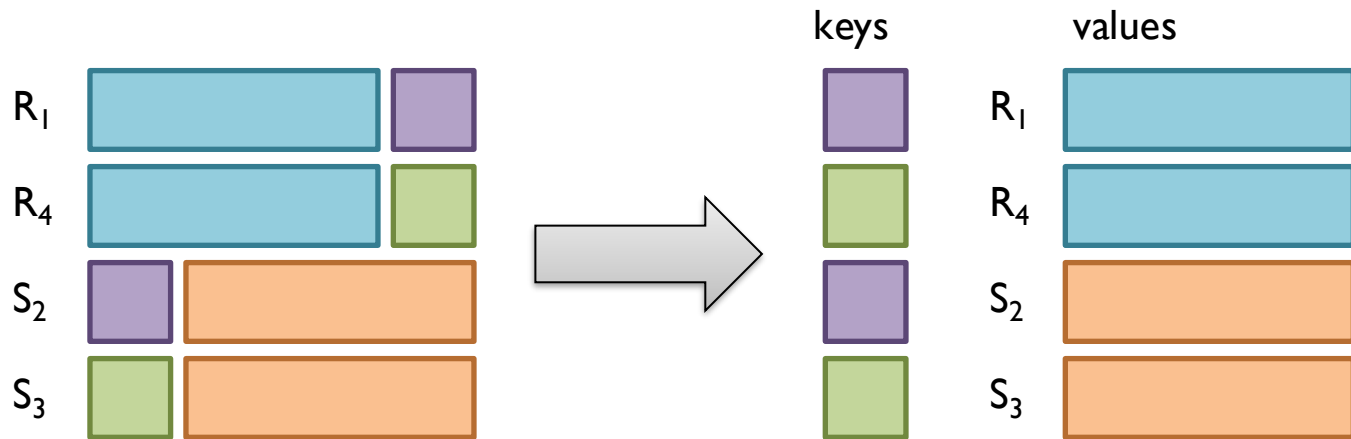
Emit tuple as value with join key as the intermediate key

Execution framework brings together tuples sharing the same key

Perform join in reducer

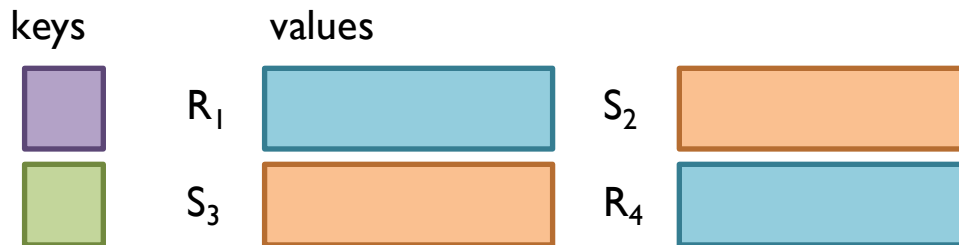
Reduce-side Join

Map



Remember to “tag” the tuple
as being from R or S...

Reduce



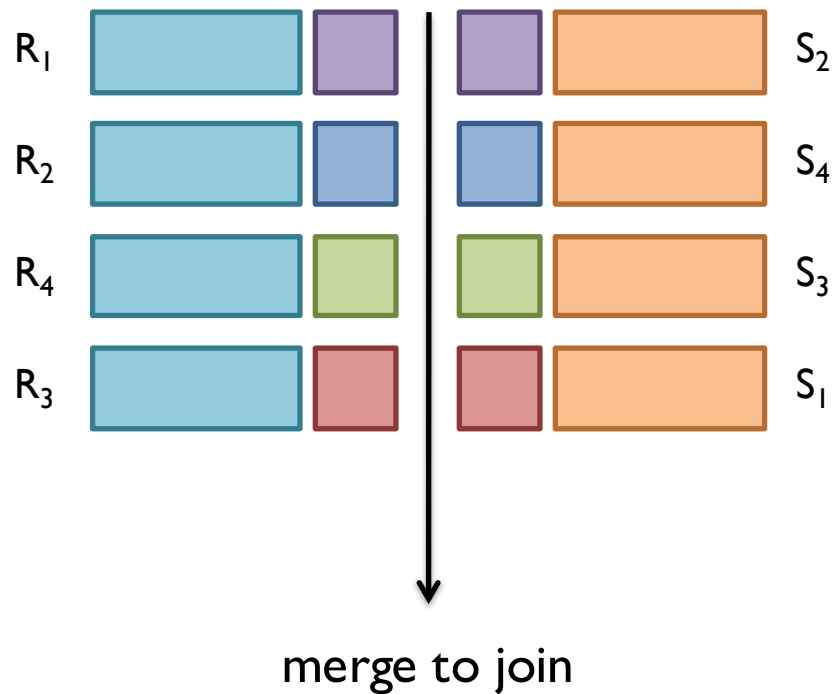
Note: no guarantee if R is going to come first or S

More precisely, an inner join: What about outer joins?

Map-side Join

aka sort-merge join

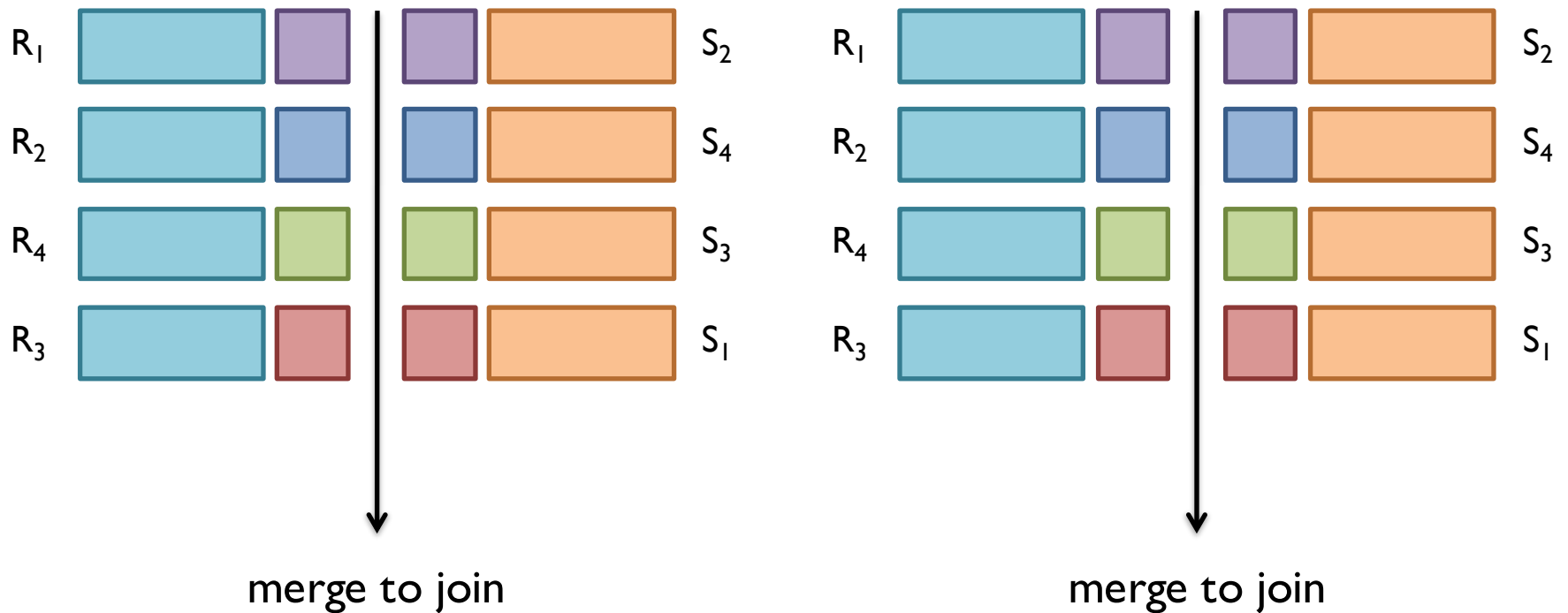
Assume two datasets are sorted by the join key:



Map-side Join

aka sort-merge join

Assume two datasets are sorted by the join key:



How can we parallelize this? Co-partitioning

Map-side Join

aka sort-merge join

Works if...

Two datasets are co-partitioned
Sorted by join key

MapReduce implementation:

Map over one dataset, read from other corresponding partition

Co-partitioned, sorted datasets: realistic to expect?
With proper setup, Spark automatically optimizes!

Hash Join

aka broadcast join, replicated join

Basic idea:

Load one dataset into memory in a hashmap, keyed by join key
Read the other dataset, probe for join key

Works if...

$R \ll S$ and R fits into memory

Implementation:

Distribute R to all machines (e.g., broadcast in Spark)
Map over S , each mapper loads R in memory and builds the hashmap
For every tuple in S , probe join key in R

Hash Join Variants

Co-partitioned variant:

R and S co-partitioned (but not sorted)?

Only need to build hashmap on the corresponding partition

Striped variant:

R too big to fit into memory?

Divide R into $R_1, R_2, R_3, \dots$ s.t. each R_n fits into memory

Perform hash join: $\forall n, R_n \bowtie S$

Take the union of all join results

Use a global key-value store:

Load R into memcached (or Redis)

Probe global key-value store for join key

Which join to use?

hash join > map-side join > reduce-side join

Limitations of each?

Hash join: memory

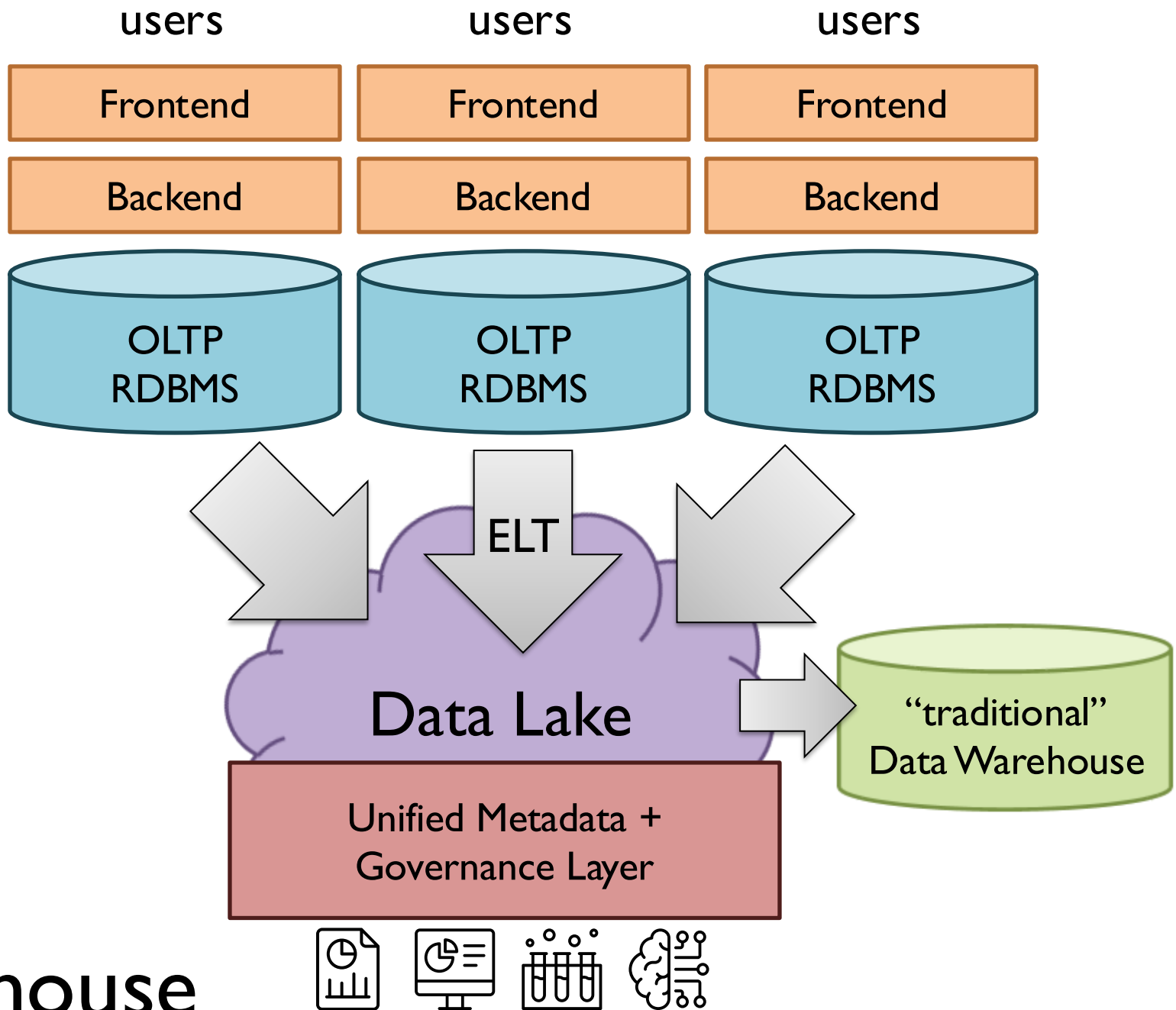
Map-side join: sort order and partitioning

Reduce-side join: general purpose

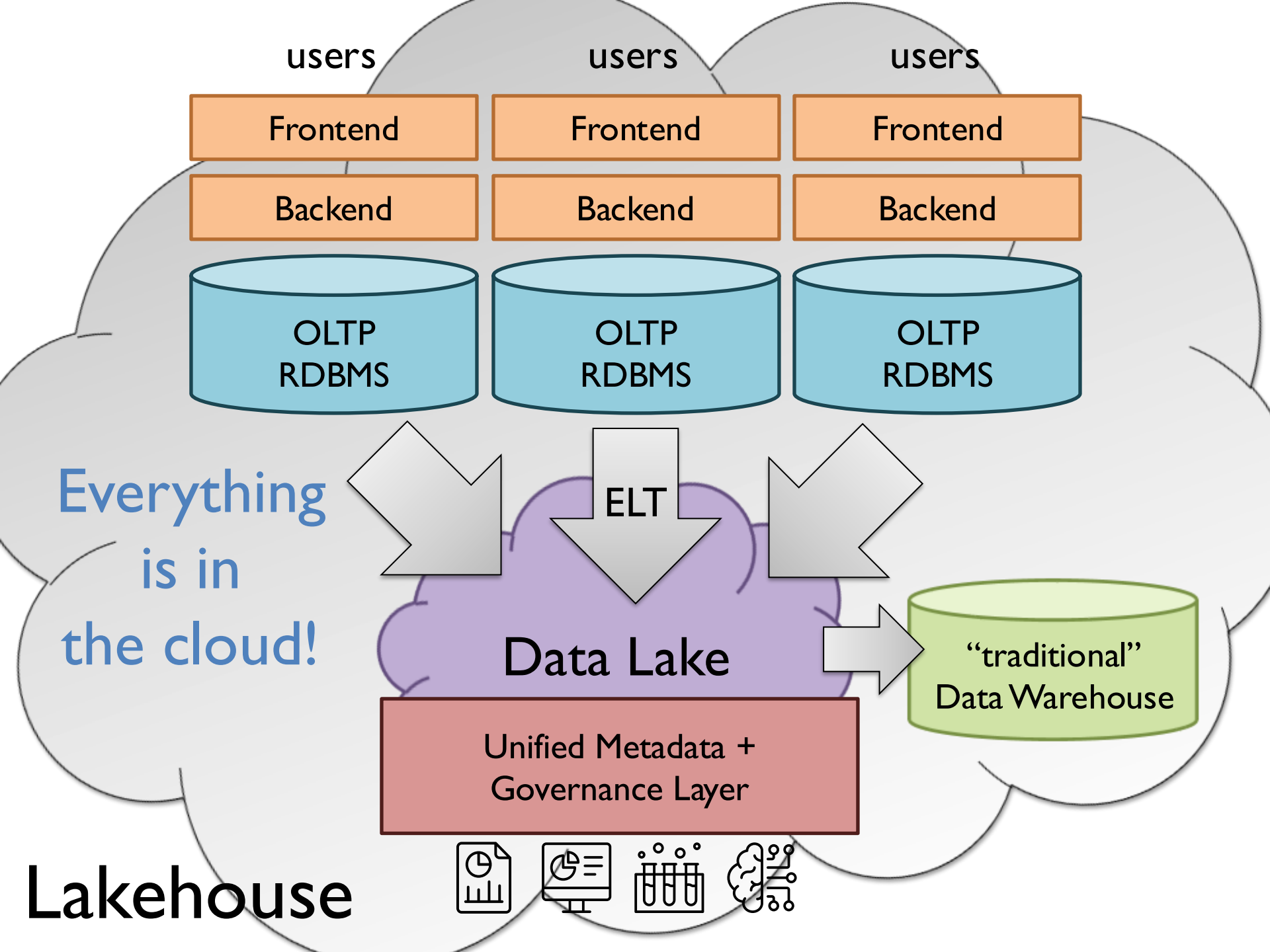
Who decides?

Technical Deep Dives

Implementation of distributed group by
... implications for algorithm design
The importance of partitioning
Case study in relational joins



Lakehouse



In the cloud, does any of this matter?

The cloud is just another abstraction!

Pros

You don't have to worry about it.

You don't need to know what's going on.



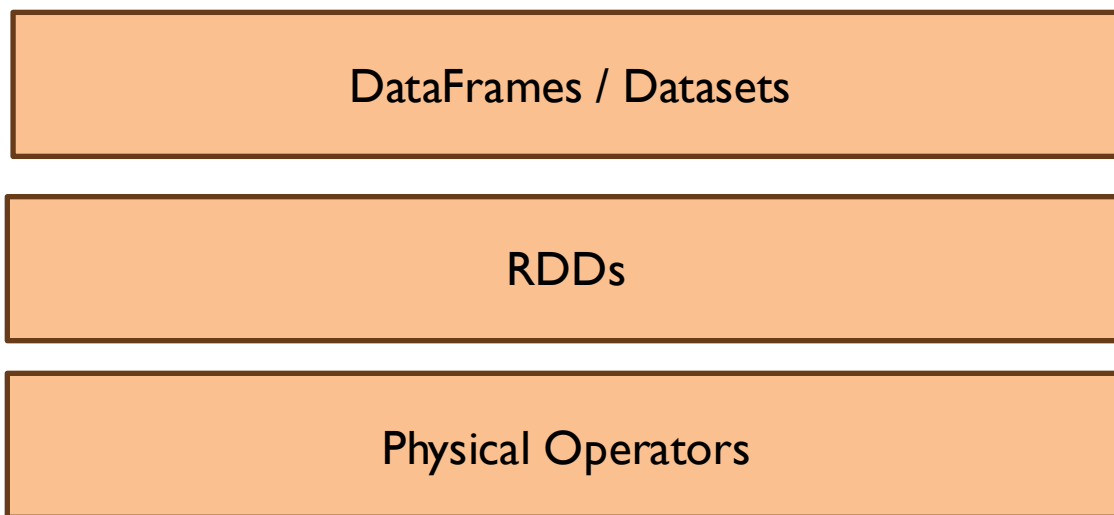
Cons

You can't worry about it (even if you wanted to).

You don't know what's going on (even if you wanted to).



Spark Stack



What level are we operating in?

